

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 5

Course: Operating Systems

Topic: The File System with Your Own Hands

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 5 · The File System with Your Own Hands

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 5 (Track A: The File System with Your Own Hands, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.9-2-cachyos)
- **Shell:** GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)
- **Filesystem (Root):** Btrfs (subvolume `/@`, zstd compressed)
- **Workspace:** `/home/ilie/Documents/SO/os-lab5` (`~/os-lab5`)

Purpose of the Work

1. Inspect the Linux virtual file system hierarchy, distinguishing standard directory layouts from system links and device nodes.
2. Examine inode structures and link counters (`i_nlink`), tracing the operational differences between hard links and symbolic links during target removal.
3. Contrast logical file length (`st_size`) with actual physical block allocation (`st_blocks`) on disk using sparse files.
4. Verify Linux Discretionary Access Control (DAC) permission evaluation on regular files versus directories (`r` vs `x` bit semantics), special permission bits (`setuid`, `sticky`), and default process masks (`umask`).
5. Query storage hardware layout and pseudo-filesystems (`procfs`), retrieving runtime hardware metrics directly from kernel-exposed interfaces.

Part 1. The Tree and the File Types

1. Root Hierarchy and Path Inspection

Checking the root directory and inspecting distinct filesystem nodes:

```
❯ ls /
bin   dev  Games  lib    mnt  path  root  sbin  sys  usr
boot  etc  home   lib64  opt  proc  run   srv   tmp  var

❯ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root    7 Oct 12  2025 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 11 02:07 /dev/null
drwxr-xr-x  1 root root 4844 Oct 12 15:23 /etc
-rw-r--r--  1 root root 2395 Sep 28 15:43 /etc/passwd
drwxrwxrwt 19 root root  560 Oct 12 15:33 /tmp
```

Figure 1.1 – Root hierarchy and system file type inspection

2. Device Nodes and File Classification

Examining `/dev` device nodes and identifying file types via the `file` utility:

```
❯ ls -l /dev | head -10
total 0
crw-r--r--   1 root root    10,   235 Oct 11 02:07 autofs
drwxr-xr-x   2 root root          220 Oct 12 15:18 block
drwxr-xr-x   2 root root           60 Oct 12 15:18 bsg
crw-rw----   1 root disk    10,   234 Oct 11 02:07 btrfs-control
drwxr-xr-x   3 root root           60 Oct 11 02:07 bus
drwxr-xr-x   2 root root         4660 Oct 12 15:18 char
crw-------   1 root root     5,     1 Oct 11 02:07 console
```

```
lrwxrwxrwx   1 root root           11 Oct 11 02:07 core -> /proc/kcore
drwxr-xr-x   6 root root          120 Oct 12 15:18 cpu

❯ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, ...
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
```

Figure 1.2 – Device nodes and character versus block classification in /dev

3. Space Utilization: `/var/log` vs `/usr`

Measuring directory disk consumption:

```
❯ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
51M /var/log
27G /usr
```

Figure 1.3 – Comparing disk utilization of /var/log versus /usr

Observe (Part 1 Answers)

1. **Type of each path from the first character in `ls -l`:**
   - `/bin`: `l` → Symbolic link.
   - `/dev/null`: `c` → Character special device.
   - `/etc`: `d` → Directory.
   - `/etc/passwd`: `-` → Regular file.
   - `/tmp`: `d` → Directory (with sticky bit `t`).
2. **What is `/bin` on this system?** `/bin` is a symbolic link pointing to `usr/bin`. This reflects the standard Arch Linux / CachyOS UsrMerge architecture, where all core binaries reside unified under `/usr/bin`, and `/bin` remains strictly as a backward-compatibility pointer.
3. **Character device and block device in `/dev`:**
   - Character device: `/dev/null` (`crw-rw-rw-`, major 1, minor 3) or `/dev/console` (`crw-------`, major 5, minor 1). Transmits unbuffered character streams directly.
   - Block device: `/dev/nvme0n1p2` (`brw-rw----`, major 259, minor 2) or `/dev/sda` (`brw-rw----`, major 8, minor 0). Transmits data in fixed addressable blocks through kernel cache/buffers.
4. **Comparison between `/var/log` and `/usr`:** `/usr` is 27 GB, while `/var/log` is 51 MB. `/usr` is larger by roughly 26.95 GB (approximately 530 times larger). This is because `/usr` stores system-wide executable binaries, libraries, toolchains, desktop assets, and documentation, whereas `/var/log` contains only text logs and journal records.

Part 2. Inodes and Links

1. Inode Identification and File Names

Creating a test file and reading its filesystem metadata:

```
❯ echo hello > notes.txt
❯ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 0,56    Inode: 5310683     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/    ilie)   Gid: ( 1000/    ilie)
Access: 2026-10-12 15:37:00.231912724 +0300
Modify: 2026-10-12 15:37:00.233648308 +0300
Change: 2026-10-12 15:37:00.233648308 +0300
 Birth: 2026-10-12 15:37:00.231912724 +0300

❯ ls -li notes.txt
5310683 -rw-r--r-- 1 ilie ilie 6 Oct 12 15:37 notes.txt
```

Figure 2.1 – Inode identification and link counter inspection

2. Hard Link Creation and Target Unlinking

Linking an additional directory entry to the existing inode and removing the original entry:

```
❯ ln notes.txt hard.txt
❯ ls -li notes.txt hard.txt
5310683 -rw-r--r-- 2 ilie ilie 6 Oct 12 15:37 hard.txt
5310683 -rw-r--r-- 2 ilie ilie 6 Oct 12 15:37 notes.txt

❯ rm notes.txt
❯ cat hard.txt ; ls -li hard.txt
hello
5310683 -rw-r--r-- 1 ilie ilie 6 Oct 12 15:37 hard.txt
```

Figure 2.2 – Inode sharing across hard links and survival after target removal

3. Symbolic Link Behavior on Broken Reference

Creating a symlink, reading its content, and deleting the referenced target:

```
❯ ln -s hard.txt soft.txt
❯ ls -li hard.txt soft.txt
5310683 -rw-r--r-- 1 ilie ilie 6 Oct 12 15:37 hard.txt
5310684 lrwxrwxrwx 1 ilie ilie 8 Oct 12 15:37 soft.txt -> hard.txt

❯ cat soft.txt
hello

❯ rm hard.txt
❯ cat soft.txt
cat: soft.txt: No such file or directory

❯ ls -l soft.txt
lrwxrwxrwx 1 ilie ilie 8 Oct 12 15:37 soft.txt -> hard.txt
```

Figure 2.3 – Symbolic link creation and broken reference detection

4. Directory Link Counter Progression

Testing directory link counters upon subdirectory creation:

```
❯ mkdir d ; ls -ld d
drwxr-xr-x 1 ilie ilie 0 Oct 12 15:37 d

❯ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 1 ilie ilie 4 Oct 12 15:37 d
```

Figure 2.4 – Subdirectory creation and directory link count progression on ext4/tmpfs

Note on Btrfs vs Traditional Unix Filesystems: The current workspace resides on a Btrfs subvolume. Modern filesystems like Btrfs do not update directory `st_nlink` upon subdirectory creation (reporting static link count 1) to prevent costly locking across large B-trees. Running the identical test on a standard POSIX filesystem (`tmpfs` / `ext4`) reveals the classic behavior:

```
❯ mkdir -p /tmp/d_test && cd /tmp/d_test
❯ mkdir d ; ls -ld d
drwxr-xr-x 2 ilie ilie 40 Oct 12 15:32 d

❯ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 ilie ilie 80 Oct 12 15:32 d
```

Figure 2.5 – Btrfs subvolume directory link count behavior

5. Logical File Size versus Allocated Disk Blocks

Comparing apparent byte size with allocated storage:

```
❯ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 ilie ilie 2 Oct 12 15:37 one.txt
4.0K    one.txt

❯ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 ilie ilie 1.0G Oct 12 15:37 big.img
0   big.img

❯ rm big.img
```

Figure 2.6 – Sparse file creation contrasting logical length with allocated disk blocks

Observe (Part 2 Answers)

1. **Which number in `stat` is the inode, and which tells how many names the file has?**
   - Inode number: `Inode: 5310683`.
   - Name counter: `Links: 1` (or `2` when `hard.txt` was linked). In the kernel `stat` structure, these map to `st_ino` and `st_nlink`.
2. **Why could you still read `hard.txt` after `rm notes.txt`? What does `rm` really remove?** `rm` invokes the `unlink()` system call. It does not erase physical data blocks or delete the inode immediately. It only removes the directory entry `notes.txt` from the containing directory and decrements `st_nlink` from 2 to 1. Because `hard.txt` still holds a reference pointing directly to inode 5310683, the inode remains valid and all data blocks remain accessible.
3. **Why did `cat soft.txt` fail after removing `hard.txt`? How big is `soft.txt` and why?**
   - `soft.txt` is an independent inode containing only the string pathname `"hard.txt"`. When `cat soft.txt` is invoked, the kernel attempts pathname resolution by following the target string to locate `hard.txt`. Because that path no longer exists, resolution fails with `ENOENT` (*No such file or directory*).
   - `soft.txt` is exactly 8 bytes because the path string `"hard.txt"` is 8 ASCII characters long. A symlink's file size in `ls -l` is the length of its target path string.
4. **Where does each link come from (2 initially, 4 with two subdirectories)?**
   - Initial 2 links:
     1. The parent directory's entry referencing `d` (`parent/d`).
     2. The directory's own self-reference `.` inside `d` (`d/.`).
   - 4 links after creating `d/x` and `d/y`:
     3. The parent directory entry `..` inside `d/x` (`d/x/..` → `d`).
     4. The parent directory entry `..` inside `d/y` (`d/y/..` → `d`).
5. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0?**
   - `one.txt` contains 2 bytes (`"x\n"`). The filesystem allocates storage in discrete block units (typically 4096 bytes). Since a non-empty file requires at least one physical block on disk, `du` reports 4.0K.
   - `big.img` was created with `truncate -s 1G`. This updates only the logical file length field (`i_size`) in the inode table without writing actual data blocks. Because filesystems support sparse files, no storage blocks are allocated for unwritten holes, resulting in 0 physical disk usage.

Part 3. Permissions

1. File Permissions, DAC Evaluation, and umask

Testing permission masks on regular files:

```
❯ echo secret > s.txt
❯ chmod 600 s.txt ; ls -l s.txt
```

```
-rw------- 1 ilie ilie 7 Oct 12 15:37 s.txt

❯ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

❯ chmod u+r s.txt ; cat s.txt
secret

❯ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
drwxr-xr-x 1 ilie ilie 0 Oct 12 15:37 newdir
-rw-r--r-- 1 ilie ilie 0 Oct 12 15:37 new.txt
```

Figure 3.1 – DAC permission verification and umask mask calculation

2. Directory Permissions: Read (r) vs Execute (x)

Predicting and testing access restrictions on directories:

```
❯ mkdir box ; echo hi > box/f

# Test A: Read-only directory (u=r,go=)
❯ chmod u=r,go= box
❯ ls box
f
❯ cat box/f
cat: box/f: Permission denied

# Test B: Execute-only directory (u=x,go=)
❯ chmod u=x,go= box
❯ ls box
ls: cannot open directory 'box': Permission denied
❯ cat box/f
hi

❯ chmod 755 box
```

Figure 3.2 – Contrast between directory read permission and execute traversal permission

3. Special Bits (SetUID, Sticky) and Protected Files

Examining binary privilege escalations and system shadow files:

```
❯ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 88984 Jul 31 22:01 /usr/bin/passwd
drwxrwxrwt 19 root root   560 Oct 12 15:33 /tmp

❯ cat /etc/shadow
cat: /etc/shadow: Permission denied

❯ ls -l /etc/shadow
-rw------- 1 root root 1124 Sep 28 15:43 /etc/shadow
```

Figure 3.3 – SetUID and Sticky bit inspection on privileged executables and directories

Observe (Part 3 Answers)

1. **Translate 600, 644, 755, and 700 into `rwx` strings with realistic use cases:**
   - `600` → `rw-------`: Read/write by owner only. Used for confidential files such as private SSH keys (`~/.ssh/id_ed25519`) or database credential configs.
   - `644` → `rw-r--r--`: Read/write by owner, read-only for group and others. Standard for public files such as web assets (`index.html`) or `/etc/passwd`.
   - `755` → `rwxr-xr-x`: Full owner permissions, read and traverse/execute for everyone else. Standard for system binaries (`/usr/bin/ls`) and shared directories (`/usr/local`).

- `700` → `rwx------`: Full owner access, zero access for group and others. Used for private user directories like `~/.ssh` or `/root`.

2. **With `chmod 000`, the owner cannot read the file. Why does ownership not help?** Linux DAC checks process credentials against the inode in strict order. If the effective UID of the process matches the file owner UID, the kernel evaluates *only* the user permission bits. When user bits are `---`, access is rejected immediately with `EACCES` without falling back to group or other permissions. Ownership gives the right to change permissions (`chmod`), but does not bypass DAC permission bits during `read()` or `open()`.

3. **What is your `umask`, and how does it explain the permissions of `new.txt` and `newdir`?**
   - Current umask is `0022`.
   - Base mode for newly created files is `0666` (`rw-rw-rw-`). Masked: `0666 & ~0022 = 0644` (`-rw-r--r--`).
   - Base mode for newly created directories is `0777` (`rwxrwxrwx`). Masked: `0777 & ~0022 = 0755` (`drwxr-xr-x`).

4. **Explain the `box` experiment (`r` vs `x` on directories):**
   - Directory `r` (read) allows reading the directory contents (list of file names) via `getdents64()`. It allows `ls box` to print `f`.
   - Directory `x` (execute / search) allows directory traversal: looking up inodes, querying file metadata, and opening files inside the path.
   - When `box` has only `r`, the shell can list `"f"`, but cannot traverse into the directory to resolve the inode of `box/f`, causing `cat box/f` to fail with *Permission denied*.
   - When `box` has only `x`, `ls box` fails because reading the directory directory stream is prohibited, but `cat box/f` succeeds because the kernel can traverse the directory and open `box/f`.

5. **Meaning of `s` in `/usr/bin/passwd` and `t` in `/tmp`:**
   - The `s` bit on `/usr/bin/passwd` is the **SetUID (Set User ID)** flag. When executed by an unprivileged user, the process runs with the effective UID of the file's owner (`root`). This is required because changing a password requires updating `/etc/shadow`, which is writable only by root (`-rw------- 1 root root`).
   - The `t` bit on `/tmp` is the **Sticky Bit** (restricted deletion flag). In a directory writable by all users (`0777`), the sticky bit prevents unprivileged users from deleting or renaming files owned by others.

Part 4. Disks and the Kernel as Files

1. Storage Topology and Mount Inspection

Inspecting local disk partitions and root mount options:

```
❯ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    1  28.9G  0 disk
├─sda1        8:1    1  28.8G  0 part
└─sda2        8:2    1    32M  0 part
zram0       253:0    0  19.4G  0 disk [SWAP]
nvme0n1     259:0    0 238.5G  0 disk
├─nvme0n1p1 259:1    0     4G  0 part /boot
└─nvme0n1p2 259:2    0 234.5G  0 part /var/log
                                      /var/tmp
                                      /var/cache
                                      /home
                                      /srv
                                      /root
                                      /
nvme1n1     259:3    0 476.9G  0 disk
└─nvme1n1p1 259:4    0 476.9G  0 part /mnt/storage
```

```
❯ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p2  235G  147G   87G  63% /

❯ mount | grep " / "
/dev/nvme0n1p2 on / type btrfs
↪  (rw,noatime,compress=zstd:1,ssd,discard=async,space_cache=v2,subvolid=256,subvol=/@)
```

Figure 4.1 – Root filesystem storage topology and Btrfs subvolume mount parameters

2. Kernel Hardware Parameters via `/proc`

Extracting processor, memory, and runtime process information:

```
❯ grep -c processor /proc/cpuinfo
4

❯ head -3 /proc/meminfo
MemTotal:       20338252 kB
MemFree:         1765864 kB
MemAvailable:   12811044 kB

❯ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   100191
Ngid:   0

❯ ls -l /proc/self/fd
total 0
lrwx------ 1 ilie ilie 64 Oct 12 15:37 0 -> /dev/pts/3
lrwx------ 1 ilie ilie 64 Oct 12 15:37 1 -> /dev/pts/3
lrwx------ 1 ilie ilie 64 Oct 12 15:37 2 -> /dev/pts/3
lr-x------ 1 ilie ilie 64 Oct 12 15:37 3 -> /proc/100184/fd
```

Figure 4.2 – CPU topology and runtime process status via /proc virtual filesystem

Observe (Part 4 Answers)

1. **Disk Partition Layout:**

```
[nvme0n1] 238.5 GB NVMe SSD
 ├── [nvme0n1p1]  4.0 GB  --> Mounted at /boot (vfat EFI system partition)
 └── [nvme0n1p2] 234.5 GB --> Mounted at / as Btrfs (subvolume /@, zstd:1)
                              [Also hosts subvolumes /@home, /@var/log, etc.]

[nvme1n1] 476.9 GB Secondary NVMe SSD
 └── [nvme1n1p1] 476.9 GB --> Mounted at /mnt/storage

[zram0] 19.4 GB RAM Swap Device
[sda]   28.9 GB Removable Flash Storage
```

2. **Root Filesystem Usage:** The root filesystem on `/dev/nvme0n1p2` has 235 GB capacity, 147 GB used, 87 GB available, and is at **63% utilization**.

3. **CPU Cores and RAM:**
   - CPU logical core count: **4 cores** (`grep -c processor /proc/cpuinfo`).
   - Memory capacity: **20,338,252 kB** (~20.34 GB / ~19.4 GiB physical RAM) with **12.81 GB** available.

4. **Program Name in `/proc/self/status`:** The program name displayed is `head` (`Name: head`). `/proc/self` is a dynamic symbolic link maintained by the kernel that always dereferences to the PID of the calling process. Since `head` is the command reading `/proc/self/status`, `/proc/self` resolves to the `head` process itself.

Part 5. Stretch Tasks

Stretch (a): Magic Numbers in File Headers

Verifying header bytes that permit `file` to detect file formats regardless of extension:

```
❯ gzip -c /etc/passwd > notes.gz
❯ file notes.gz
notes.gz: gzip compressed data, was "passwd", last modified: Mon Sep 28 12:43:28 2026, from Unix, original
↪  size modulo 2^32 2395

❯ xxd notes.gz | head -1
00000000: 1f8b 0808 f060 ba6a 0003 7061 7373 7764  .....`.j..passwd
```

Figure 5.1 – Identifying file signatures and magic numbers with xxd

Analysis: The first two bytes `1f 8b` (hexadecimal `0x1F8B`) represent the magic number mandated by RFC 1952 for Gzip streams. The third byte (`0x08`) specifies the compression method (`DEFLATE`). The `file` utility matches these bytes against its magic database rather than relying on the `.gz` filename suffix.

Stretch (b): Searching for SetUID Binaries

Auditing privileged executables installed across the root filesystem:

```
❯ find / -xdev -perm -4000 -type f 2>/dev/null | head -12
/usr/bin/ksu
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/fusermount3
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/su
```

Figure 5.2 – System-wide audit of SetUID executables

Analysis: Beyond standard Unix utilities shown in lecture slides (`passwd`, `su`, `sudo`), the system includes modern privileged helpers such as `pkexec` (Polkit authorization framework) and `fusermount3` (enabling unprivileged users to mount FUSE filesystems like browser profile RAM-caches).

Stretch (c): Unlinking an Open File

Demonstrating kernel reference counting on unlinked file descriptors:

```
❯ echo "open file test" > bigfile
❯ exec 3< bigfile
❯ rm bigfile
❯ ls -l /proc/$$/fd | grep bigfile
lr-x------ 1 ilie ilie 64 Oct 12 15:33 3 -> '/home/ilie/Documents/SO/os-lab5/bigfile (deleted)'
❯ exec 3<&-
```

Figure 5.3 – Preserved inode access and delayed block reclamation on open unlinked files

Analysis: When `rm bigfile` executes, the directory entry is removed from the dentry tree, but the link in `/proc/$$/fd/3` shows the file marked as `(deleted)`. The kernel does not deallocate the inode or free physical disk blocks while active open file descriptors exist. The space is reclaimed only after closing descriptor 3 (`exec 3<&-`), when the inode reference count drops to 0.

Conclusions

This lab explored Linux virtual file system primitives: 1. **Name Decoupling from Inodes:** File names are merely entries in directory files associating strings with numeric inodes. Hard links add directory references to existing inodes, while `unlink()` only decrements link counts, freeing blocks when link count and open handle counts hit zero. 2. **Symbolic Link Indirection:** Symlinks are standalone files storing path strings whose size strictly matches the path string character count. Target deletion leaves broken symlinks without affecting the link itself. 3. **Storage Granularity and Sparse Allocations:** Small files consume at least one filesystem block (4.0 KiB), while files initialized with logical holes (`truncate`) allocate zero physical blocks until written. 4. **Discretionary Access Control Semantics:** On directories, `r` permits listing entries, while `x` permits path traversal and inode resolution. SetUID enables unprivileged tools like `passwd` to run with elevated root privileges, while Sticky bits on `/tmp` prevent deletion of foreign files. 5. **Kernel Pseudo-Filesystem Introspection:** `/proc` exposes hardware metrics, CPU topology, and process-specific file descriptor tables dynamically generated in memory.